KH정보교육원

애플리케이션 테스트 관리

목차

Part 1. 애플리케이션 테스트 케이스 설계하기

Chapter 1. 애플리케이션 테스트 케이스 작성 4

Chapter 2. 애플리케이션 테스트 시나리오 작성 13

Part 2. 애플리케이션 통합 테스트하기

Chapter 3. 애플리케이션 통합 테스트 수행 18

Chapter 4. 애플리케이션 테스트 결과 분석 26

Chapter 5. 애플리케이션 개선 조치사항 작성 31

Chapter 6. 애플리케이션 성능 분석 35

Chapter 7. 애플리케이션 성능 개선 39

Part 1. 애플리케이션 테스트 케이스 설계하기

Chapter 1. 애플리케이션 테스트 케이스 작성

1 응용 소프트웨어의 유형 및 특성 이해

응용 소프트웨어는 불특정 일반인에게 필요한 공통의 기능을 제공하는 상용 소프트웨어와 특정 사용자의 요구사항을 구현하기 위한 서비스 제공 소프트웨어로 구분할 수 있다.

1.1 상용 소프트웨어의 특성 및 유형

상업적 목적이나 판매를 목적으로 생산되나, 홍보를 위한 무료 소프트웨어도 포함할 수 있으며, 산업의 특성에 따라 산업 범용 소프트웨어와 산업 특화 소프트웨어로 구분된다.

(1) 산업 범용 소프트웨어

산업 범용 소프트웨어는 다시 시스템 소프트웨어, 미들웨어, 응용 소프트웨어로 구분 할 수 있으며, 임베디드/리얼타임 운영체제, 웹 애플리케이션 서버, 영상 관련 소프트 웨어 등이 대표적인 유형이다.

(가) 시스템 소프트웨어 운영체제(임베디드/리얼타임, 모바일, PC/서버 등), DBMS(DataBase Management System), 데이터 통합, 프로그래밍 언어, 스토리지 소프트웨어, 소프트웨어공학 도 구, 가상화 소프트웨어, 시스템 보안 소프트웨어 등이 있다.

(나) 미들웨어 웹 애플리케이션 서버, 실시간 데이터 처리, 연계 통합 솔루션, 분산 병렬 처리, 네 트워크 관리, 시스템 관리, 클라우드 서비스, 접근 제어 소프트웨어 등이 있다

(다) 응용 소프트웨어

영상 인식/분석, 영상 코덱/스트리밍, 영상 저작/편집/합성, 3D 스캐닝/프린팅, 가상 시뮬레이션, 콘텐츠 보호/관리/유통, 정보검색, 음성 처리, 오피스웨어 소프트웨어 등이 있다.

(2) 산업 특화 소프트웨어

특정한 산업 분야에서 요구하는 기능만을 구현하기 위한 목적의 소프트웨어로 자동차, 항공, 조선, 건설, 패션 의류, 농업, 의료, 국방, 공공 분야 등을 지원하는 소프트웨어가 존재한다.

1.2. 서비스 제공 소프트웨어의 특성 및 유형

개발된 소프트웨어의 판매가 아닌 특정한 사용자의 요구사항만을 구현함을 목적으로 생산 되며, 신규 기능 개발, 기존에 개발된 기능 개선, 사용자 요구 기능의 추가 개발, 구현된 시스템의 통합을 위한 소프트웨어의 개발 등으로 유형을 분류할 수 있다.

(1) 신규 개발 소프트웨어

새로운 서비스 제공을 목적으로 개발되며, 일반적으로 초기 개발 단계, 확장 단계, 기 능 고도화 단계 등으로 진행된다.

(2) 기능 개선 소프트웨어

기존 서비스 기능에서 사용자 편의성 개선, 응답 속도 개선, 화면 UI(User Interface) 개선, 업무 프로세스 개선 등의 목적으로 개발되는 소프트웨어이다.

(3) 추가 개발 소프트웨어

기존 서비스 제공 시스템에 업무 환경의 변화, 산업 환경의 변화, 법/제도의 개정 등으 로 인해 새로운 기능을 추가로 개발하는 소프트웨어를 말한다.

(4) 시스템 통합 소프트웨어

각각 별도로 서비스되는 시스템을 원스톱(One-Stop) 서비스 제공을 위해 업무 기능 및 데이터 등을 통합하여 개발하는 소프트웨어이다.

2 소프트웨어 테스트의 이해

2.1 소프트웨어 테스트의 개념

소프트웨어 테스트란 구현된 응용 애플리케이션이나 시스템이 사용자가 요구하는 기능의 동작과 성능, 사용성, 안정성 등을 만족하는지 확인하기 위하여 소프트웨어의 결함을 찾아 내는 활동이다.

2.2. 소프트웨어 테스트의 필요성

(1) 오류 발견 관점

프로그램에 잠재된 오류를 발견하고 이를 수정하여 올바른 프로그램을 개발하는 활동이다.

(2) 오류 예방 관점

프로그램 실행 전에 코드 리뷰, 동료 검토, 인스펙션 등을 통해 오류를 사전에 발견하 는 예방 차원의 활동이다.

(3) 품질 향상 관점

사용자의 요구사항 및 기대 수준을 만족하도록 반복적인 테스트를 거쳐 제품의 신뢰 도를 향상하는 품질 보증 활동이다.

2.3. 소프트웨어 테스트의 기본 원칙

(1) 소프트웨어 테스트의 원리

(가) 테스팅은 결함이 존재함을 밝히는 활동이다.
테스팅은 소프트웨어의 잠재적인 결함을 줄일 수 있지만, 결함이 발견되지 않아도 결함이 없다고 증명할 수 없음을 나타낸다.

(나) 완벽한 테스팅은 불가능하다.
무한 경로, 무한 입력 값, 무한 시간이 소요되어 완벽하게 테스트할 수 없으므로 리스크 분석과 우선순위를 토대로 테스트에 집중할 것을 의미한다.

(다) 테스팅은 개발 초기에 시작해야 한다.

애플리케이션의 개발 단계에 테스트를 계획하고 SDLC(Software Development Life Cycle)의 각 단계에 맞춰 전략적으로 접근하는 것을 고려하라는 뜻이다.

(라) 결함 집중(Defect Clustering)

애플리케이션 결함의 대부분은 소수의 특정한 모듈에 집중되어 존재한다.

(마) 살충제 패러독스(Presticide Paradox)

동일한 테스트 케이스로 반복 실행하면 결함을 발견할 수 없으므로 주기적으로 테 스트 케이스를 리뷰하고 개선해야 한다.

(바) 테스팅은 정황(Context)에 의존한다.

정황과 비즈니스 도메인에 따라 테스트를 다르게 수행하여야 한다.

(사) 오류-부재의 궤변(Absence of Errors Fallacy)

사용자의 요구사항을 만족하지 못하는 오류를 발견하고 그 오류를 제거하였다 해도, 해당 애플리케이션의 품질이 높다고 말할 수 없다.

(2) 소프트웨어 테스트 프로세스

일반적인 테스트 프로세스는 테스트 계획, 테스트 분석, 테스트 디자인, 테스트 케이스 및 시나리오 작성, 테스트 수행, 테스트 결과 평가 및 리포팅의 절차로 이루어진다.



[그림 1-1] 테스트 프로세스

(3) 소프트웨어 테스트 산출물

(가) 테스트 계획서

테스트 목적과 범위 정의, 대상 시스템 구조 파악, 테스트 수행 절차, 테스트 일정, 조직의 역할 및 책임 정의, 종료 조건 정의 등 테스트 수행을 계획한 문서

(나) 테스트 케이스

테스트를 위한 설계 산출물로, 응용 소프트웨어가 사용자의 요구사항을 준수하는지 확인하기 위해 설계된 입력 값, 실행 조건, 기대 결과로 구성된 테스트 항목의 명세서

(다) 테스트 시나리오

테스트 수행을 위한 여러 개의 테스트 케이스의 집합으로 테스트 케이스의 동작 순서를 기술한 문서이며, 테스트를 위한 절차를 명세한 문서

(라) 테스트 결과서

테스트 결과를 정리한 문서로 테스트 프로세스를 리뷰하고, 테스트 결과를 평가하고 리포팅하는 문서

3 소프트웨어 테스트의 유형

3.1. 프로그램 실행 여부

(1) 정적 테스트

프로그램 실행 없이 소스 코드의 구조를 분석하여 논리적으로 검증하는 테스트로 **인 스펙션, 코드 검사, 워크스루 등이 있다.**

(2) 동적 테스트

프로그램의 실행을 요구하는 테스트로 화이트박스 테스트와 블랙박스 테스트가 있다.

3.2. 테스트 기법

(1) 화이트박스 테스트

프로그램의 내부 로직(수행 경로 구조, 루프 등)을 보면서 테스트를 수행한다.

(2) 블랙박스 테스트

프로그램의 외부 사용자 요구사항 명세를 보면서 테스트, 주로 구현된 기능을 테스트 한다.

3.3. 테스트에 대한 시각

(1) 검증(Verification)

제품의 생산 과정을 테스트한다(Are we building the product right?). 올바른 제품을 생산하고 있는지 검증하는 것을 의미한다.

(2) 확인(Validation)

생산된 제품의 결과를 테스트한다.(Are we building the right product?) 생산된 제품이 정상적으로 동작하는지 확인하는 것을 의미한다.

3.4. 테스트 목적

(1) 회복(Recovery) 테스트

시스템에 고의로 실패를 유도하고 시스템이 정상적으로 복귀하는지 테스트한다.

(2) 안전(Security) 테스트

불법적인 소프트웨어가 접근하여 시스템을 파괴하지 못하도록 소스코드 내의 보안적 인 결함을 미리 점검하는 테스트이다.

(3) 강도(Stress) 테스트

시스템에 과다 정보량을 부과하여 과부하 시에도 시스템이 정상적으로 작동되는지를 검증하는 테스트이다.

(4) 성능(Performance) 테스트

사용자의 이벤트에 시스템이 응답하는 시간, 특정 시간 내에 처리하는 업무량, 사용자 요구에 시스템이 반응하는 속도 등을 테스트한다.

(5) 구조(Structure) 테스트

시스템의 내부 논리 경로, 소스코드의 복잡도를 평가하는 테스트이다.

(6) 회귀(Regression) 테스트

변경 또는 수정된 코드에 대하여 새로운 결함 발견 여부를 평가하는 테스트이다.

(7) 병행(Parallel) 테스트

변경된 시스템과 기존 시스템에 동일한 데이터를 입력 후 결과를 비교하는 테스트이다.

3.5. 테스트 종류

(1) 명세 기반 테스트

주어진 명세를 빠짐없이 테스트 케이스로 구현하고 있는지 확인하는 테스트를 말하며, 동등 분할, 경계 값 분석, 유한상태 기계기반, 결정 테이블, 정형명세 기반, 유스케이스, 페어와이즈, 직교배열 테스트 등이 있다.

(2) 구조 기반 테스트

소프트웨어 내부 논리 흐름에 따라 테스트 케이스를 작성하고 확인하는 테스트를 말 하며, 구문 기반, 결정 기반, 조건 기반, 조건결정 기반, 변경조건 결정 기반, 멀티조건 기반 커버리지 테스트 등이 있다.

(3) 경험 기반 테스트

유사 소프트웨어나 유사 기술 평가에서 테스터의 경험을 토대로 한, 직관과 기술 능력 을 기반으로 수행하는 테스트를 말한다.

4 테스트 케이스와 테스트 오라클의 이해

4.1. 테스트 케이스의 개념

명세 기반 테스트의 설계 산출물로, 특정한 프로그램의 일부분 또는 경로에 따라 수행하 거나, 특정한 요구사항을 준수하는지 확인하기 위해 설계된 입력 값, 실행 조건, 기대 결 과로 구성된 테스트 항목의 명세서를 말한다.

4.2. 테스트 케이스 작성

테스트 케이스의 정확성, 재사용성, 간결성 보장을 위해 아래의 절차에 따라 작성한다.

(1) 테스트 계획 검토 및 자료 확보

테스트 대상 프로젝트 범위와 접근 방법 이해를 위하여 테스트 계획을 재검토한다. 테스트 대상 시스템 자료와 정보를 확보하여, 시스템 요구사항과 기능 명세서를 검토한다.

(2) 위험 평가 및 우선순위 결정

결함 해결에 있어 상대적 중요성을 지니며 테스트의 초점을 결정한다.

(3) 테스트 요구사항 정의

시스템 요구사항, 테스트 대상 재검토, 테스트할 특성, 조건, 기능을 식별 및 분석한다.

(4) 테스트 구조 설계 및 테스트 방법 결정

테스트 케이스의 일반적 형식을 결정하고, 테스트 케이스 분류 방법을 결정한다. 테스트 절차, 장비, 도구, 테스트 문서화 방법을 결정한다.

(5) 테스트 케이스 정의

각 요구사항에 대해 테스트 케이스를 작성하고, 입력 값, 실행 조건, 예상 결과를 기술한다.

(6) 테스트 케이스 타당성 확인 및 유지 보수

기능 또는 환경 변화에 따라 테스트 케이스를 갱신하고, 테스트 케이스의 유용성을 검 토한다.

4.3. 테스트 오라클의 개념

(1) 테스트 오라클 정의

테스트의 결과가 참인지 거짓인지를 판단하기 위해서 사전에 정의된 참 값을 입력하 여 비교하는 기법 및 활동을 말한다.

(2) 테스트 오라클 유형

(가) 참(True) 오라클

모든 입력 값에 대하여 기대하는 결과를 생성함으로써 발생된 오류를 모두 검출할 수 있는 오라클이다.

(나) 샘플링(Sampling) 오라클

특정한 몇 개의 입력 값에 대해서만 기대하는 결과를 제공해 주는 오라클이다.

(다) 휴리스틱(Heuristic) 오라클

샘플링 오라클을 개선한 오라클로, 특정 입력 값에 대해 올바른 결과를 제공하고, 나머지 값들에 대해서는 휴리스틱(추정)으로 처리하는 오라클이다.

(라) 일관성 검사(Consistent) 오라클

애플리케이션 변경이 있을 때, 수행 전과 후의 결과 값이 동일한지 확인하는 오라클이다.

(3) 오라클 적용 방안

참 오라클은 주로 항공기, 임베디드, 발전소 소프트웨어 등 미션 크리티컬한 업무에 적용 하고, 샘플링/추정 오라클은 일반, 업무용, 게임, 오락 등의 일반적인 업무에 적용한다.

Chapter 2. 애플리케이션 테스트 시나리오 작성

1 테스트 시나리오의 이해

1.1. 테스트 시나리오의 개념

(1) 테스트 시나리오의 정의

테스트 수행을 위한 여러 테스트 케이스의 집합으로서, 테스트 케이스의 동작 순서를 기술한 문서이며 테스트를 위한 절차를 명세한 문서이다.

(2) 테스트 시나리오의 필요성

테스트 수행 절차를 미리 정함으로써 설계 단계에서 중요시되던 요구사항이나 대안 흐름과 같은 테스트 항목을 빠짐없이 테스트하기 위함이다.

1.2. 테스트 시나리오 작성 시 유의점

(1) 테스트 시나리오 분리 작성

테스트 항목을 하나의 시나리오에 모두 작성하지 않고, 시스템별, 모듈별, 항목별 테스 트 시나리오를 분리하여 작성한다.

(2) 고객의 요구사항과 설계 문서 등을 토대로 테스트 시나리오를 작성한다.

(3) 각 테스트 항목은 식별자 번호, 순서 번호, 테스트 데이터, 테스트 케이스, 예상 결 과, 확인 등의 항목을 포함하여 작성한다.

2 테스트 환경 구축의 이해

2.1. 테스트 환경 구축의 개념

개발된 응용 소프트웨어가 실제 운영 시스템에서 정상적으로 작동하는지 테스트할 수 있 도록 하기 위하여 실제 운영 시스템과 동일 또는 유사한 사양의 하드웨어, 소프트웨어, 네 트워크 등의 시설을 구축하는 활동이다

2.2. 테스트 환경 구축의 유형

(1) 하드웨어 기반의 테스트 환경 구축

서버 장비(WAS 서버, DBMS 서버), 클라이언트 장비(노트북 또는 PC), 네트워크(내부 LAN 또는 공용 인터넷 라인) 장비 등의 장비를 설치하는 작업이다.

(2) 소프트웨어 기반의 테스트 환경 구축

구축된 하드웨어 환경에 테스트할 응용 소프트웨어를 설치하고 필요한 데이터를 구축 하는 작업이다.

(3) 가상 시스템 기반의 테스트 환경 구축

물리적으로 개발 환경 및 운영 환경과 별개로 독립된 테스트 환경을 구축하기 힘든 경우에는, 가상 머신(Virtual Machine) 기반의 서버 또는 클라우드 환경을 이용하여 테 스트 환경을 구축하고, 네트워크는 VLAN 과 같은 기법을 이용하여 논리적 분할 환경을 구축할 수 있다.

3 테스트 데이터 및 테스트 조건의 이해

3.1. 테스트 데이터의 개념

컴퓨터의 동작이나 시스템의 적합성을 시험하기 위해 특별히 개발된 데이터 집합으로서 프로그램의 기능을 하나씩 순번에 따라 확실하게 테스트할 수 있도록 조건을 갖춘 데이터 를 말한다.

(1) 테스트 데이터의 필요성

테스트 수행 시 잘못된 데이터를 사용하면 잘못된 결과가 도출되어 시간을 낭비하고 비용만 소진하는 결과가 나온다. 따라서 테스트를 효율적으로 운용하고 데이터의 기밀 을 유지하며 신뢰 및 예측 가능한 테스트를 위해 테스트 데이터의 준비가 필요하다.

(2) 테스트 데이터의 유형

선행된 연산에 의해 얻어진 실제 데이터와 인위적으로 만들어진 가상의 데이터로 구 분된다.

(3) 테스트 데이터 준비

실제 데이터는 연산에 의해 준비하거나 실제 운영 데이터를 복제하여 준비할 수 있으 며, 가상의 데이터는 스크립트를 통해서 생성할 수 있다.

3.2. 테스트 조건의 개념

(1) 테스트 시작 조건

테스트 계획의 수립, 사용자 요구사항에 대한 테스트 명세의 작성, 투입 조직 및 참여 인력의 역할과 책임의 정의, 테스트 일정의 확정, 테스트 환경의 구축 등이 완료되었 을 때 테스트를 시작하도록 조건을 정의할 수 있으며, 단계별 또는 회차별 테스트 수 행을 위해 모든 조건을 만족하지 않아도 테스트를 시작할 수 있다.

(2) 테스트 종료 조건

테스트의 종료는 정상적인 테스트를 모두 수행한 경우, 차기 일정의 도래로 테스트 일 정이 만료되었을 경우, 테스트에 소요되는 비용을 모두 소진한 경우 등 업무 기능의 중요도에 따라 조건을 달리 정할 수 있다.

(3) 테스트 성공과 실패의 판단 기준

기능 및 비기능 테스트 시나리오에 기술된 예상 결과를 만족하면 성공으로 아니라면 실패로 판단할 수 있으며, 또는 동일한 데이터 또는 이벤트를 중복하여 테스트하여도 여전히 이전 테스트와 같은 결과가 나올 때 성공으로 판단할 수도 있다.

Part 2. 애플리케이션 통합 테스트하기

Chapter 3. 애플리케이션 통합 테스트 수행

1 통합 테스트 수행 방법

1.1. 개요

(1) 통합 테스트의 개념

애플리케이션 통합 테스트는 소프트웨어 각 모듈 간의 인터페이스 관련 오류 및 결함 을 찾아내기 위한 체계적인 테스트 기법이다.

(2) 통합 테스트의 목적

단위 테스트가 끝난 모듈 또는 컴포넌트 단위의 프로그램이 설계 단계에서 제시한 애 플리케이션과 동일한 구조와 기능으로 구현된 것인지를 확인하는 것이다.

(3) 통합 테스트 수행 방법의 분류

일반적으로 점증적인 방법과 비점증적인 방식으로 나눌 수 있다. 비점증적인 빅뱅 방 식은 모든 컴포넌트를 사전에 통합하여 전체 프로그램을 한꺼번에 테스트하는 것을 말하며, 점증적인 방법은 다시 상향식 통합과 하향식 통합 방식으로 구분할 수 있다.

1.2. 하향식 통합(Top Down)

(1) 방식

메인 제어 모듈(프로그램)로부터 아래 방향으로 제어의 경로를 따라 이동하면서 하향 식으로 통합하면서 테스트를 진행하며, 메인 제어 모듈에 통합되는 하위 모듈과 최하 위 모듈은 '깊이-우선' 또는 '너비-우선' 방식으로 통합된다.

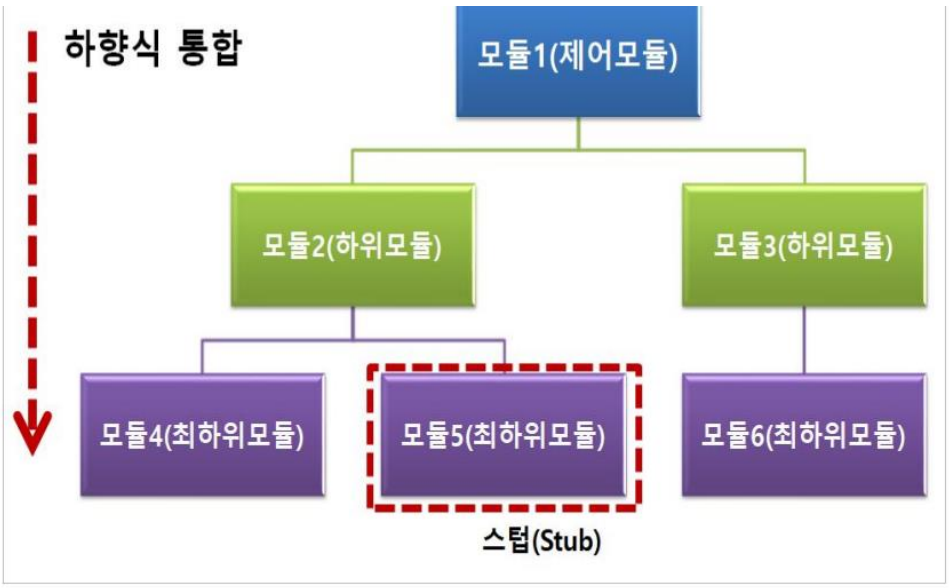
(2) 수행 단계

(가) 메인 제어 모듈은 작성된 프로그램을 사용하고, 아직 작성되지 않은 하위 제어 모듈 및 모든 하위 컴포넌트를 대신하여 더미 모듈인 스텁(Stub)을 개발한다.

(나) 깊이-우선 방식 또는 너비-우선 방식에 따라, 하위 모듈인 스텁이 한 번에 하나씩 실제 모듈로 대체된다.

(다) 각 모듈 또는 컴포넌트를 통합하면서 테스트가 수행된다.

(라) 테스트가 완료되면 스텁이 실제 모듈 또는 컴포넌트로 작성된다.



[그림 2-1] 하향식 방식의 통합 테스트

1.3. 상향식 통합(Bottom Up)

(1) 방식

애플리케이션 구조에서 최하위 레벨의 모듈 또는 컴포넌트로부터 위쪽 방향으로 제어 의 경로를 따라 이동하면서 구축과 테스트를 시작한다.

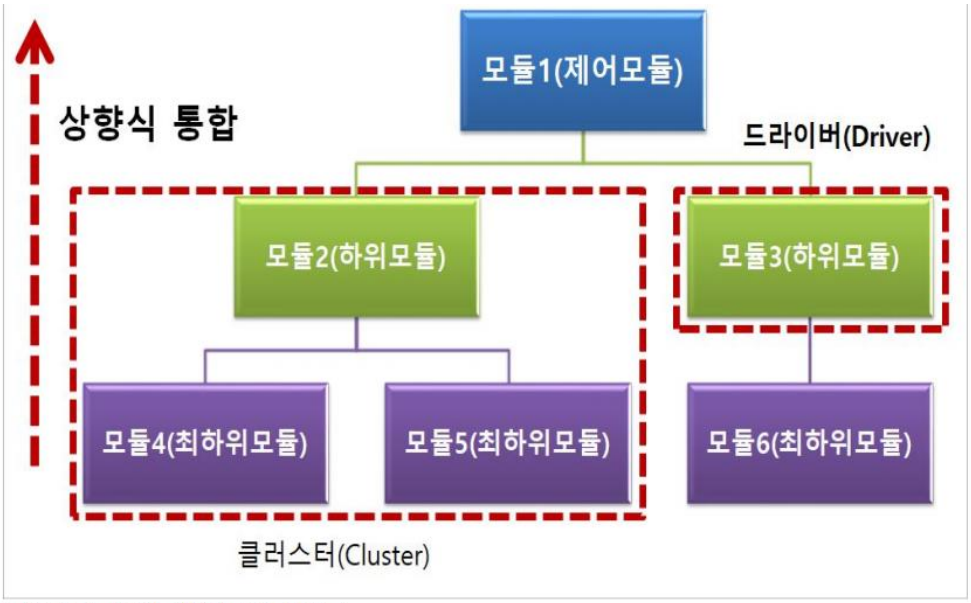
(2) 수행 단계

(가) 최하위 레벨의 모듈 또는 컴포넌트들이 하위 모듈의 기능을 수행하는 클러스터 (Cluster)로 결합된다.

(나) 상위의 모듈에서 데이터의 입력과 출력을 확인하기 위한 더미 모듈인 드라이버 (Driver)를 작성한다.

(다) 각 통합된 클러스터 단위를 테스트한다.

(라) 테스트가 완료되면 각 클러스터들은 프로그램의 위쪽으로 결합되며, 드라이버 는 실제 모듈 또는 컴포넌트로 대체된다.



[그림 2-2] 상향식 방식의 통합 테스트

1.4. 회귀 테스팅(Regression Testing)

통합 테스트가 완료된 후에 변경된 모듈이나 컴포넌트가 있다면 새로운 오류 여부를 확인 하기 위해 회귀 테스트를 수행할 수 있다.

(1) 정의

(가) 통합 테스트 과정에서 오류를 제거하거나 수정한 프로그램이 새로운 형태의 오 동작이나 오류를 일으킬 수 있다.

(나) 회귀 테스트는 모듈이나 컴포넌트의 변화로 인해 의도하지 않은 오류가 생기지 않았음을 보증하기 위해 반복 테스트하는 것을 말한다.

(2) 회귀 테스트 케이스 선정 방법

(가) 모든 애플리케이션의 기능을 수행할 테스트 케이스의 대표적인 샘플을 도출한다.

(나) 변경에 의한 영향도가 가장 높은 애플리케이션 기능에 집중한 추가적인 테스트 케이스를 도출한다.

(다) 실제 수정이 발생한 모듈 또는 컴포넌트에서부터 시행하는 테스트 케이스를 도 출한다.

2 테스트 자동화 도구

2.1. 테스트 자동화

(1) 테스트 자동화의 개념

테스트 자동화란 테스트 도구를 활용하여 반복적인 테스트 작업을 스크립트 형태로 구현함으로써, 테스트 시간 단축과 인력 투입 비용을 최소화하는 한편, 쉽고 효율적인 테스트를 수행할 수 있는 방법이다.

(2) 테스트 도구의 장점

(가) 반복되는 테스트 데이터 재입력 작업의 자동화

(나) 사용자 요구 기능의 일관성 검증에 유리

(다) 테스트 결과 값에 대한 객관적인 평가 기준 제공

(라) 테스트 결과의 통계 작업과 그래프 등 다양한 표시 형태 제공

(마) UI 가 없는 서비스의 경우에도 정밀한 테스트 가능

(3) 테스트 도구의 단점

(가) 도구 도입 후 도구 사용 방법에 대한 교육 및 학습이 필요

(나) 도구를 프로세스 단계별로 적용하기 위한 시간, 비용, 노력이 필요

(다) 상용 도구의 경우 고가, 유지 관리 비용이 높아 추가 투자가 필요

2.2. 테스트 자동화 도구 유형

(1) 정적 분석 도구(Static Analysis Tools)

(가) 정적 분석 도구는 만들어진 애플리케이션을 실행하지 않고 분석하는 방법이다.

(나) 대부분의 경우 소스 코드에 대한 코딩 표준, 코딩 스타일, 코드 복잡도 및 남은 결함을 발견하기 위하여 사용한다.

(다) 테스트를 수행하는 사람이 작성된 소스 코드에 대한 이해를 바탕으로 도구를 이용해서 분석하는 것을 말한다.

(2) 테스트 실행 도구(Test Execution Tools)

이 도구는 테스트를 위해 작성된 스크립트를 실행한다. 작성된 스크립트는 각 스크립 트마다 특정 데이터와 테스트 수행 방법을 포함하고 있으며, 데이터 주도 접근 방식과 키워드 주도 접근 방식으로 나눌 수 있다.

(가) 데이터 주도 접근 방식

테스트 데이터를 스프레드시트에 저장하고, 이 데이터를 읽고 실행할 수 있도록 한 다. 다양한 테스트 데이터를 이용하여 동일한 테스트 케이스를 반복해서 실행할 수 있으며, 스크립트 언어에 익숙지 않은 테스터도 미리 작성된 스크립트에 테스트 데 이터만 추가하여 쉽게 테스트를 수행할 수 있게 된다.

(나) 키워드 주도 접근 방식

일반적으로 테스트를 수행할 동작을 나타내는 키워드와 테스트 데이터를 스프레드 시트에 저장한다. 이 방식에서는 키워드를 이용하여 테스트 수행 동작을 정의할 수 있으며, 테스트 대상 애플리케이션의 특성에 맞추어 키워드에 대해 테일러링 (Tailoring)을 수행할 수 있다.

(3) 성능 테스트 도구(Performance Test Tools)

성능 테스트 도구는 애플리케이션의 처리량, 응답 시간, 경과 시간, 자원 사용률에 대해 가상의 사용자를 생성하고 테스트를 수행함으로써 성능 목표를 달성하였는지를 확인하 는 도구이다. 일반적으로 시스템 테스트 단계에 성능 테스트 계획을 설계하고 실행하고 있으며, 테스트 결과를 해석할 수 있는 성능 전문가의 도움이 필요한 경우도 있다.

(4) 테스트 통제 도구(Test Control Tools)

테스트 통제 도구에는 테스트 계획 및 관리를 위한 테스트 관리 도구, 테스트 수행에 필요한 데이터와 도구를 관리하는 형상 관리 도구, 테스트에서 발생한 결함에 대해 관 리하거나 협업을 지원하기 위한 결함 추적/관리 도구 등이 있다. 또한 조직의 요구사 항에 최적화된 형태의 정보를 생성, 관리하기 위하여 스프레드시트 등 다른 도구들과 연계하여 사용할 수도 있다.

(5) 테스트 장치(Test Harness)

(가) 정의

애플리케이션 컴포넌트 및 모듈을 테스트하는 환경의 일부분으로, 테스트를 지원하 기 위한 코드와 데이터를 말하며, 단위 또는 모듈 테스트에 사용하기 위해 코드 개 발자가 작성한다.

(나) 구성요소

1) 테스트 드라이버(Test Driver) 테스트 대상 하위 모듈을 호출하고, 파라미터를 전달하고, 모듈 테스트 수행 후 의 결과를 도출하는 등 상향식 테스트에 필요하다.

2) 테스트 스텁(Test Stub) 제어 모듈이 호출하는 타 모듈의 기능을 단순히 수행하는 도구로 하향식 테스 트에 필요하다.

3) 테스트 슈트(Test Suites) 테스트 대상 컴포넌트나 모듈, 시스템에 사용되는 테스트 케이스의 집합을 말한다.

4) 테스트 케이스(Test Case) 입력 값, 실행 조건, 기대 결과 등의 집합을 말한다.

5) 테스트 스크립트(Test Script) 자동화된 테스트 실행 절차에 대한 명세를 말한다.

6) 목 오브젝트(Mock Object) 사용자의 행위를 조건부로 사전에 입력해 두면, 그 상황에 예정된 행위를 수행 하는 객체를 말한다.

(6) 테스트 자동화 도구

테스트 자동화 도구는 휴먼 에러(Human Error)를 줄이고, 테스트에 소요되는 비용과 시간을 절감하며, 테스트 품질을 향상할 수 있는 도구이다. 테스트 계획, 테스트 분석/설계, 테스트 수행, 테스트 통제 등의 테스트 활동에 따라 각각 요구사항 관리, 테스트 케이스 생성, 커버리지 분석, 정적/동적 테스트 수행, 성능 테스트, 모니터링, 형상 관리, 테스트 관리, 결함 추적/관리 등을 지원하는 테스트 도구가 있다.

<표 2-1> 테스트 단계별 테스트자동화 도구

테스트 단계	자동화 도구	도 구 설 명
테스트 계획	요구사항 관리	고객 요구사항 정의 및 요구사항 관리를 지원
테스트 분석/설계	테스트케이스 생성	테스트 기법에 따른 테스트 케이스 작성과 테스트 데이터 생성을 지원
테스트 수행	테스트 자동화	기능 테스트와 UI 테스트 등 단위 테스트 및 통합테스트를 지원
	정적 분석	코딩표준, 런타임 오류 등을 검증
	동적 분석	대상시스템 시뮬레이션을 통한 오류 검출
	성능 테스트	부하생성기 등을 이용하여 가상 사용자를 생성하고, 시스템의 처리능력을 측정하는 도구
	모니터링	시스템 자원(CPU, Memory 등)의 상태 확인 및 분석 지원
테스트 관리	커버리지 측정	테스트 완료 후 테스트 충분성 여부 검증 지원
	형상관리	테스트 수행에 필요한 도구, 데이터 및 문서 관리
	결함 추적/관리	테스트에서 발생한 결함 추적 및 관리 활동 지원

Chapter 4. 애플리케이션 테스트 결과 분석

1 테스트 결과 분석

1.1 소프트웨어 결함

소프트웨어의 결함을 말할 때 에러(Error), 결함(Defect), 결점(Fault), 버그(Bug), 실패 (Failure)와 같은 용어가 사용되며, 이런 용어들의 정의를 다음과 같이 정리할 수 있다.

(1) 에러(Error)/오류

에러는 결함(Defect)의 원인이 되는 것으로, 일반적으로 사람(소프트웨어 개발자, 분석 가 등)에 의해 생성된 실수를 말한다.

(2) 결함/결점/버그(Bug)

에러 또는 오류가 원인이 되어 소프트웨어 제품에 포함되어 있는 결함을 말하며, 이를 제 거하지 않으면 소프트웨어 제품이 실패(Failure)하거나 문제(Problem)가 발생할 수 있다.

(3) 실패/문제

소프트웨어 제품에 포함된 결함이 실행될 때 발생되는 현상을 말한다.

1.2. 테스트 완료 조건

단위 테스트, 통합 테스트, 시스템 테스트, 인수 테스트 등 각 단계별 테스트를 언제 어떤 상황에서 종료할 것인지를 결정하는 것이다. 이러한 완료 조건은 프로젝트 특성에 따라 일정, 비용, 조직 등에 제약이 있으므로, 최적의 완료 조건을 계획하여야 한다.

1.3. 테스트 리포팅

(1) 테스트 결과 정리

모든 테스트가 완료되면, 테스트 계획과 테스트 케이스 설계부터 단계별 테스트 시나 리오, 테스트 결과까지 모두 포함된 문서를 작성한다.

(2) 테스트 요약 문서

테스트 계획, 소요 비용, 테스트 결과로 판단 가능한 대상 소프트웨어의 품질 상태를 포함한 문서를 작성한다.

(3) 품질 상태

품질 상태는 정량화된 품질 지표인 테스트 성공률, 테스트 커버리지, 발생한 결함의 수와 결함의 중요도 등이 포함된다.

(4) 테스트 결과서

테스트 결과서는 결함과 관련한 사항을 중점적으로 기록하며, 결함의 내용 및 자원 정 보(CPU, Memory, Network 사용률과 로그 정보 등), 결함의 재현 순서를 상세하게 기록 한다.

(5) 테스트 실행 절차 및 평가

단계별 테스트 종료 시 테스트 실행 절차를 리뷰하고 결과에 대한 평가를 수행하고, 그 결과에 따라 실행 절차를 최적화하여 다음 테스트에 적용한다.

2 결함 관리

2.1. 테스트 결함 관리

(1) 개요

테스트 결함 관리란 각 단계별 테스트 수행 후 발생한 결함의 재발 방지를 위해, 유사 결함 발견 시 처리 시간 단축을 위해 결함을 추적하고 관리하는 활동이다.

(2) 결함 추적 관리 활동

단위 테스트, 통합 테스트, 시스템 테스트, 운영 테스트의 단계별 착수 기준과 입력물, 그리고 종료 조건 및 산출물에 대해 다음과 같이 정리할 수 있다.

<표 2-5> 결함 추적 관리 활동

테스트 단계	착수 기준	입력물	종료 조건 / 산출물
단위 테스트	- 해당 사항 없음.	- 테스트 활동 로그 - 결함 관리 대장	- 종료 조건 : 심각도 상위 수준의 결함 해결 여부 확인 - 산출물 : 결함 관리 대장, 테스트 활동 로그, 테스트 결과 보고서
통합 테스트	- 설계 문서 결함 발견 - 통합 테스트 평가 완료	- 테스트 활동 로그 - 결함 관리 대장	
시스템 테스트	- 요구사항 명세서 결함 발견 - 시스템 테스트 평가 완료	- 테스트 활동 로그 - 결함 관리 대장	
운영 테스트	- 요구사항 명세서 결함 발견 - 운영 테스트 평가 완료	- 테스트 활동 로그 - 결함 관리 대장	

2.2. 결함 관리 프로세스

(1) 에러 발견

요구사항 분석, 설계, 테스트 실행 중 에러가 발견될 경우, 테스트 전문가와 프로젝트 팀과 논의한다.

(2) 에러 등록

결함 관리 대장에 발견된 에러를 등록한다.

(3) 에러 분석

등록된 에러가 단순 에러인지 아니면 실제 결함인지 분석한다.

(4) 결함 확정

등록된 에러가 실제 결함으로 확정될 경우, 결함 확정 상태로 설정한다.

(5) 결함 할당

결함을 해결할 담당자를 지정하여 결함을 할당하고 결함 할당 상태로 설정한다.

(6) 결함 조치

결함에 대해 수정 활동을 수행하고, 수정이 완료된 경우 결함 조치 상태로 설정한다.

(7) 결함 조치 검토 및 승인

수정이 완료된 결함에 대해 확인 테스트를 수행하고, 정상적으로 결함 조치가 완료된 경우 결함 조치 완료 상태로 설정한다.

3 결함 추이 분석

3.1. 결함 추이 분석

(1) 개요

테스트 완료 후 발견된 결함의 결함 관리 측정 지표의 속성 값들을 분석하고, 향후 애 플리케이션의 어떤 모듈 또는 컴포넌트에서 결함이 발생할지를 추정하는 작업이다.

(2) 분석 유형

결함 추이 분석에는 결함의 분포 분석, 결함 추세 분석, 결함 에이징 분석 등의 유형이 있다.

3.2. 결함 관리 측정 지표

(1) 결함 분포

각 애플리케이션 모듈 또는 컴포넌트의 특정 속성에 해당하는 결함의 수를 측정하여 결함의 분포를 분석할 수 있다.

(2) 결함 추세

테스트 진행 시간의 흐름에 따른 결함의 수를 측정하여 결함 추세를 분석할 수 있다.

(3) 결함 에이징

등록된 결함에 대해 특정한 결함 상태의 지속 시간을 측정하여 분석할 수 있다.

Chapter 5. 애플리케이션 개선 조치사항 작성

1 테스트 커버리지(Test Coverage)

1.1 개요

테스트 커버리지란 주어진 테스트 케이스에 의해 수행되는 소프트웨어의 테스트 범위를 측 정하는 테스트 품질 측정 기준이며, 테스트의 정확성과 신뢰성을 향상시키는 역할을 한다.

1.2. 기능 기반 커버리지

테스트 대상 애플리케이션의 전체 기능을 모수로 설정하고, 실제 테스트가 수행된 기능의 수를 측정하는 방법이다. 기능 기반 테스트 커버리지는 100% 달성을 목표로 하며, 일반적 으로 UI 가 많은 시스템의 경우 화면 수를 모수로 사용할 수도 있다.

1.3. 라인 커버리지(Line Coverage)

애플리케이션 전체 소스 코드의 Line 수를 모수로 테스트 시나리오가 수행한 소스 코드의 Line 수를 측정하는 방법이다. 단위 테스트에서는 이 라인 커버리지를 척도로 삼기도 한다.

1.4. 코드 커버리지(Code Coverage)

소프트웨어 테스트 충분성 지표 중 하나로 소스 코드의 구문, 조건, 결정 등의 구조 코드 자체가 얼마나 테스트되었는지를 측정하는 방법이다.

(1) 구문(Statement) 커버리지

코드 구조 내의 모든 구문에 대해 한 번 이상 수행하는 테스트 커버리지를 말한다.

(2) 조건(Condition) 커버리지

결정 포인트 내의 모든 개별 조건식에 대해 수행하는 테스트 커버리지를 말한다.

(3) 결정(Decision) 커버리지

결정 포인트 내의 모든 분기문에 대해 수행하는 테스트 커버리지를 말한다.

(4) 변형 조건/결정(Modified Condition/Decision) 커버리지

조건과 결정을 복합적으로 고려한 측정 방법이며, 결정 포인트 내의 다른 개별적인 조 건식 결과에 상관없이 독립적으로 전체 조건식의 결과에 영향을 주는 테스트 커버리 지를 말한다.

2 테스트 결함 식별 및 관리

2.1. 결함의 식별

(1) 단계별 결함 유입 분류

(가) 기획 시 유입되는 결함

사용자 요구사항의 표준 미준수로 인해 테스트 불가능, 요구사항 불명확/불완전/불 일치, 기타 결함이 발생할 수 있다.

(나) 설계 시 유입되는 결함

기획 단계에 유입된 결함 또는 설계 표준 미준수로 인해 테스트 불가능, 기능 설계 불명확/불완전/불일치, 기타 결함이 발생할 수 있다.

(다) 코딩 시 유입되는 결함

설계 단계에 유입된 결함 또는 코딩 표준 미준수로 인해 기능의 불일치/불완전, 데 이터 결함, 인터페이스 결함, 기타 결함이 발생할 수 있다.

(라) 테스트 부족으로 유입되는 결함

테스트 수행 시 테스트 완료 기준의 미준수, 테스트 팀과 개발 팀의 의사소통 부족, 개발자의 코딩 실수로 인한 결함이 발생할 수 있다.

(2) 결함 심각도별 분류

애플리케이션에 발생한 결함이 어떤 영향을 끼치며, 그 결함이 얼마나 치명적인지를 나타내는 척도이다.

(가) 결함 심각도 분류 사례

치명적(Critical) 결함, 주요(Major) 결함, 보통(Normal) 결함, 경미한(Minor) 결함, 단 순(Simple) 결함 등으로 분류할 수 있다.

(나) 결함 심각도 관리 결함

관리의 정확성과 신뢰성 향상을 위해 결함 심각도의 각 단계별로 표준화된 용어를 사용하여 정의하여야 하며, 프로젝트 및 조직 차원에서 결함 관리 활동을 수행해야 한다.

(3) 결함 우선순위

(가) 결함 심각도와 결함 우선순위

결함 우선순위는 발생한 결함이 얼마나 빠르게 처리되어야 하는지를 결정하는 척도로, 결함 심각도가 높아도 우선순위가 반드시 높은 것은 아니며, 애플리케이션의 특성에 따라 우선순위가 결정될 수 있다

(나) 결함 우선순위

표현 사례 결정적(Critical), 높음(High), 보통(Medium), 낮음(Low) 또는 즉시 해결, 주의 요망, 대기, 개선 권고 등으로 분류할 수 있다.

2.2. 결함 관리 항목

테스트 수행 후 발견된 결함은 결함 관리 시스템에 등록하여 관리해야 하며, 등록 시 다 음 항목들은 필수로 등록한다.

(1) 결함 내용

(2) 결함 ID

(3) 결함 유형

(4) 발견일

(5) 심각도

(6) 우선순위(결함 수정의 우선순위)

(7) 시정 조치 예정일

(8) 수정 담당자

(9) 재테스트 결과

(10) 종료일

Chapter 6. 애플리케이션 성능 분석

1 애플리케이션 성능 점검의 개요

애플리케이션 성능이란 사용자의 요구 기능을 해당 애플리케이션이 최소의 자원을 사용하면 서 얼마나 빨리, 많은 기능을 수행하는가를 육안 또는 도구를 통하여 점검하는 것을 말한다.

1.1. 애플리케이션의 성능을 측정하기 위한 지표

(1) 처리량(Throughput)

애플리케이션이 주어진 시간에 처리할 수 있는 트랜잭션의 수로, 웹 애플리케이션의 경우 시간당 페이지 수로 표현하기도 한다.

(2) 응답 시간(Response Time)

사용자 입력이 끝난 후, 애플리케이션의 응답 출력이 개시될 때까지의 시간으로, 웹 애플리케이션의 경우 메뉴 클릭 시 해당 메뉴가 나타나기까지 걸리는 시간을 말한다.

(3) 경과 시간(Turnaround Time)

애플리케이션에 사용자가 요구를 입력한 시점부터 트랜잭션 처리 후 그 결과의 출력 이 완료할 때까지 걸리는 시간을 말한다.

(4) 자원 사용률(Resource Usage)

애플리케이션이 트랜잭션 처리하는 동안 사용하는 Cpu 사용량, 메모리 사용량, 네트워 크 사용량을 말한다.

1.2. 유형별 성능 분석 도구 성능 분석

도구는 애플리케이션의 성능을 점검하는 도구와 시스템 자원 사용량을 모니터링하는 도구로 분류할 수 있다

(1) 성능/부하/스트레스(Performance/Load/Stress) 점검 도구

애플리케이션의 성능 점검을 위해 가상의 사용자를 점검 도구 상에서 인위적으로 생 성한 뒤, 시스템의 부하나 스트레스를 통해 성능 측정 지표인 처리량, 응답 시간, 경과 시간 등을 점검하기 위한 도구이다.

(2) 모니터링(Monitoring) 도구

모니터링 도구는 애플리케이션 실행 시 자원 사용량을 확인하고 분석 가능한 도구로, 성능 모니터링, 성능 저하 원인 분석, 시스템 부하량 분석, 장애 진단, 사용자 분석, 용 량 산정 등의 기능을 제공하여, 시스템의 안정적 운영을 지원하는 도구이다.

2 애플리케이션 성능 저하 원인 분석

애플리케이션의 성능이 저하되는 원인은 크게 DB 연결 및 쿼리 실행, 내부적인 요인과 외 부적인 요인, 그리고 기타 환경 설정이나, 네트워크 등의 문제로 구분할 수 있다.

2.1. 데이터베이스 연결 및 쿼리 실행 시 발생되는 성능 저하 원인

일반적으로 DB 에 연결하기 위해 Connection 객체를 생성하거나 쿼리를 실행하는 애플리 케이션 로직에서 성능 저하 또는 장애가 많이 발견된다.

(1) 데이터베이스 락(DB Lock)

대량의 데이터 조회, 과도한 업데이트, 인덱스 생성 시 발생하는 현상이며, 요청한 작 업은 Lock 의 해제 시까지 대기하거나 타임아웃된다.

(2) 불필요한 데이터베이스 페치(DB Fetch)

실제 필요한 데이터보다 많은 대량의 데이터 요청이 들어올 경우, 또는 결과 세트에서 마지막 위치로 커서를 옮기는 작업이 빈번한 경우 응답 시간 저하 현상이 발생한다.

(3) 연결 누수(Connection Leak) / 부적절한 커넥션 풀 크기(Connection Pool Size)

(가) 연결 누수(Connection Leak) DB 연결과 관련한 JDBC 객체를 사용 후 종료하지 않을 경우 발생한다.
(나) 부적절한 커넥션 풀 크기(Connection Pool Size) 너무 작거나 크게 설정한 경우 성능 저하 현상이 발생할 가능성이 있다.

(4) 기타

트랜잭션이 확정(Commit)되지 않고 커넥션 풀에 반환되거나, 잘못 작성된 코드로 인해 불필요한 Commit 가 자주 발생하는 경우 성능이 저하될 가능성이 존재한다.

2.2. 내부 로직으로 인한 성능 저하 원인

(1) 웹 애플리케이션의 인터넷 접속 불량

웹 애플리케이션의 실행 시 인터넷 접속 불량으로 서버 소켓(Server Socket) 쓰기는 지 속되나, 클라이언트에서 정상적 읽기가 수행되지 않아 성능이 저하될 가능성이 있다.

(2) 특정 파일의 업로드, 다운로드로 인한 성능 저하

대량의 파일을 업로드하거나 다운로드할 경우 처리시간이 길어져 애플리케이션의 성 능이 저하될 가능성이 있다.

(3) 정상적으로 처리되지 않은 오류 처리로 인한 성능 저하

오류 처리 로직과 실제 처리 로직 부분을 분리하지 않고 코딩하거나 예외가 발생할 경우에 제대로 처리되지 않아 행이 걸리는 상황이 발생하여 성능이 저하될 가능성이 있다.

2.3. 외부 호출(HTTP, 소켓 통신)로 인한 성능 저하 원인

임의의 트랜잭션이 수행되는 동안 외부 트랜잭션(외부 호출)이 장시간 수행되거나, 타임아 웃이 일어나는 경우 성능 저하 현상이 발생할 수 있다.

2.4. 잘못된 환경 설정이나 네트워크 문제로 인한 성능 저하 원인

(1) 환경 설정으로 인한 성능 저하

스레드 풀(Thread Pool), 힙 메모리(Heap Memory)의 크기를 너무 작게 설정하면 Heap Memory Full 현상 발생으로 성능이 저하될 가능성이 있다.

(2) 네트워크 장비로 인한 성능 저하

라우터, L4 스위치 등 네트워크 관련 장비 간 데이터 전송 실패 또는 전송 지연에 따 른 데이터 손실 발생 시 애플리케이션의 성능 저하 또는 장애가 발생할 수 있다

Chapter 7. 애플리케이션 성능 개선

1 소스 코드 최적화의 이해

소스 코드 최적화는 읽기 쉽고 변경 및 추가가 쉬운 클린 코드를 작성하는 것으로, 소스 코드 품질을 위해 기본적으로 지킬 원칙과 기준을 정의하고 있다.

1.1. 나쁜 코드(Bad Code)

(1) 개요

다른 개발자가 로직(Logic)을 이해하기 어렵게 작성된 코드이다. 대표적인 사례로 처리 로직의 제어가 정제되지 않고 서로 얽혀 있는 스파게티 코드, 변수나 메소드에 대한 이름 정의를 알 수 없는 코드, 동일한 처리 로직이 중복되게 작성된 코드 등이 있다.

(2) 문제점

스파게티 코드의 경우 잦은 오류가 발생할 가능성이 있다. 소스 코드 이해가 안되어 계속 덧붙이기할 경우 코드 복잡도가 증가한다.

1.2. 클린 코드(Clean Code)

(1) 개요

잘 작성되어 가독성이 높고, 단순하며, 의존성을 줄이고, 중복을 최소화하여 깔끔하게 잘 정리된 코드를 말한다.

(2) 효과성

중복 코드 제거로 애플리케이션의 설계가 개선된다.
가독성이 높으므로 애플리케이션의 기능에 대해 쉽게 이해할 수 있다.
버그를 찾기 쉬워지며, 프로그래밍 속도가 빨라진다.

2 소스 코드 품질 분석 도구의 이해

소스 코드에 대한 코딩 스타일, 설정된 코딩 표준, 코드의 복잡도, 코드 내에 존재하는 메 모리 누수 현황, 스레드의 결함 등을 발견하기 위하여 사용하는 분석 도구이며, 정적 분석 도구와 동적 분석 도구가 있다.

2.1 정적 분석 도구

작성된 소스 코드를 실행시키지 않고, 코드 자체만으로 코딩 표준 준수 여부, 코딩 스타일 적정 여부, 잔존 결함 발견 여부를 확인하는 코드 분석 도구이다.

(1) 정적 분석 도구의 유형

사전에 결함을 발견하고 예방하는 도구, 코딩 표준 준수 여부를 분석하는 도구, 소스 코드의 복잡도를 계산하는 도구 등이 있다.

2.2. 동적 분석 도구

애플리케이션을 실행하여 코드에 존재하는 메모리 누수 현황을 발견하고, 발생한 스레드 의 결함 등을 분석하기 위한 도구이다.

<표 3-4> 소스코드 품질 분석 도구

구분	도구명	도구 설명	지원환경	개발도구 지원	홈페이지
정적 분석 도구	pmd	자바 및 타 언어 소스코드에 대한 버그, 데드코드 분석	Linux, Windows	Eclipse, NetBeans	pmd.github.io
	cppcheck	C/C++ 코드에 대한 메모리누수, 오버플로우 등 문제 분석	Windows	Eclipse, gedit	cppcheck.sourceforge.net
	SonarQube	소스코드 품질 통합 플랫폼, 플러그인 확장 가능	Cross-Platform	Eclipse	www.sonarqube.org
	checkstyle	자바 코드에 대한 코딩 표준 준수 검사 도구	Cross-Platform	Ant, Eclipse, NetBeans	checkstyle.sourceforge.net
코드 복잡도	ccm	다양한 언어의 코드 복잡도 분석 도구, Linux, Mac 환경 CLI 형태 지원	Cross-Platform	Visual Studio	github.com/jonasblunck/ccm
	cobertura	jcoverage 기반의 테스트 커버리지 측정 도구	Cross-Platform	Ant, Maven	cobertura.github.io/cobertura
동적 분석 도구	Avalanche	Valgrind 프레임워크 및 STP 기반 소프트웨어 에러 및 취약점 동적 분석 도구	Linux. Android	-	code.google.com/archive/p/avalanche/
	Valgrind	자동화된 메모리 및 쓰레드 결함 발견 분석 도구	Cross-Platform	Eclipse, NetBeans	valgrind.org